DETECTA — VIVA DEFENSE & QUICK GUIDE

Smart AI Surveillance Camera with Multi-Modal Vision & Edge IoT

Presenter: Ahmmed Waseg Bin Mizan Orgho (ID: 251031046)
Role: Software Lead | **Course:** CSE-214 | Feni University

১. ১-মিনিট এলিভেটর পিচ (ম্যামের সামনে ওপেনিং স্পিচ — মুখস্থ বলার জন্য)

বাংলা ডায়ালগ (Oral Presentation) বাংলা

*"আসসালামু আলাইকুম ম্যাম। আমাদের প্রজেক্ট **Detecta** একটি এজ-টু-ওয়ার্কস্টেশন স্মার্ট এআই সার্ভিল্যান্স ক্যামেরা। প্রজেক্টে আমার মূল দায়িত্ব ছিল **Software Lead** হিসেবে— মাল্টি-মোডাল এআই ইঞ্জিন, হাই-স্পিড স্ট্রিমিং ও ব্যাকএন্ড ডেভেলপমেন্ট।*
*আমরা ওয়্যারলেস **ESP32-CAM** থেকে ডেডিকেটেড Port 81-এ ভিডিও স্ট্রিম নিয়ে লোকাল **RTX 4050 GPU** তে ৩টি এআই মডেল একসাথে চালাচ্ছি: **YOLO11s** (২১টি অবজেক্ট), **InsightFace ArcFace 512-D** (ফেস ভেরিফিকেশন), এবং **MediaPipe Hands** (৭টি জেসচার)। এছাড়া খালি আঙুলকে কলম ভাবা রোধে আমরা নিজস্ব **Smart Spatial Pen Filter** বানিয়েছি। সম্পূর্ণ সিস্টেমটি লোকাল ড্যাশবোর্ড ও নেটিভ উইন্ডোজ অ্যাপ (**Detecta.exe**) হিসেবে কোনো ক্লাউড ছাড়াই টানা **২০–২৫+ FPS** এ চলে।"*

English Pitch (If Evaluator Prefers English) English

*"Respected Ma'am, our project **Detecta** decouples edge IoT capture from local GPU computing. I served as the **Software Lead**, developing the multi-modal AI engine, high-speed ingestion pipeline, and backend.*
*An untethered **ESP32-CAM** streams VGA MJPEG over dedicated Port 81. On our workstation GPU (RTX 4050), we execute three concurrent pipelines: **Ultralytics YOLO11s** (21 curated classes), **InsightFace ArcFace 512-D biometrics** (with 25-frame stride caching), and **MediaPipe Hands** with spatial pen disambiguation. The system sustains **20–25+ FPS** with sub-50ms latency via FastAPI and a native desktop app (**Detecta.exe**) with zero cloud dependency."*

২. সফটওয়্যার আর্কিটেকচার ও মূল ইনোভেশন (SOFTWARE LEAD CORE CONTRIBUTIONS)

১. OpenCV সকেট ডেডলক বাইপাস (`camera_stream.py`)

- **সমস্যা:** সাধারণ `cv2.VideoCapture` ওয়াই-ফাই নেটওয়ার্কের প্যাকেট জিটারে সকেট ব্লক করে ভিডিও ফ্রিজ করে ফেলে।
- **সমাধান:** পাইথনের `requests` দিয়ে মেমোরি থেকে সরাসরি JPEG SOI (`0xFFD8`) ও EOI (`0xFFD9`) বাউন্ডারি স্লাইস করে জিরো-ল্যাগ স্ট্রিমিং ও অ্যাটমিক বাফারিং নিশ্চিত করা হয়েছে।

২. YOLO11s অবজেক্ট ও কার্টন ডিটেকশন (`ai_engine.py`)

- **CUDA FP16 ইনফারেন্স:** ২১টি কিউরেটেড ক্লাস (মানুষ, ল্যাপটপ, ফোন, কলম, বই ইত্যাদি) মাত্র **১১.৪ms** লেটেন্সিতে ডিটেক্ট হয়।
- **বক্স ও পার্সেল হ্যান্ডলিং:** ক্লাস ২৪, ২৬ ও ২৮ কে টিউন করে ডেলিভারি ও স্টোরেজ কার্টন নিখুঁতভাবে ডিটেক্ট করা হয়েছে।
- **EMA ফিল্টার:** বাউন্ডিং বক্স যাতে না কাঁপে, সেজন্য Exponential Moving Average (α=0.65, TTL=6) ব্যবহার করা হয়েছে।

৩. ArcFace 512-D ও ২৫-ফ্রেম স্ট্রাইড ক্যাশিং (`ai_engine.py`)

- **Dlib বনাম ArcFace:** Dlib ১২৮-D ভেক্টর তৈরি করে এবং অ্যাঙ্গেলে ফেস মিস করে। ArcFace ৫১২-D ভেক্টর ও কোসাইন সিমিলারিটি দেয়।
- 🔥 **২৫-ফ্রেম স্ট্রাইড উদ্ভাবন:** প্রতি ফ্রেমে ফেস মডেল ৩৮ms সময় নেয়। আমরা একবার ফেস ভেরিফাই করে পরবর্তী ২৫ ফ্রেমের জন্য আইডি ক্যাশ রাখি। ফলে গড় লেটেন্সি **<২ms** এ নেমে আসে এবং FPS থাকে **২০–২৫+**!

৪. স্মার্ট পেন ডিসঅ্যাম্বিগিউয়েশন অ্যালগরিদম (`ai_engine.py`)

- **সমস্যা:** খালি তর্জনী আঙুল দেখালে এআই প্রায়ই বিভ্রান্ত হয়ে কলম (Pen) হিসেবে ডিটেক্ট করে ফেলে।
- **আমাদের সমাধান:** মিডিয়াপাইপ হ্যান্ড বাউন্ডারি চেক করা হয়। আঙুলের মাথা ছাড়িয়ে কোনো বাস্তব বস্তু অন্তত **২৫ পিক্সেল বাইরে প্রসারিত** থাকলেই কেবল তা `Pen [Held]` হিসেবে ভেরিফাই হয়; খালি আঙুল স্বয়ংক্রিয়ভাবে বাতিল হয়।

৫. সিঙ্গল-পাস ক্যাশিং ও উইন্ডোজ অ্যাপ (`backend.py` / `Detecta.exe`)

- **FastAPI Single-Pass Cache:** ব্যাকগ্রাউন্ড থ্রেডে ফ্রেমটি একবারই এনকোড হয়ে মেমোরিতে থাকে; সব ক্লায়েন্ট এক ক্যাশ থেকে স্ট্রিম নেয় (CPU লোড ৭০% কম)।
- **নেটিভ ডেস্কটপ অ্যাপ:** Microsoft Edge WebView2 ইঞ্জিন দিয়ে তৈরি স্ট্যান্ডঅ্যালোন `Detecta.exe` অ্যাপ।

৩. হার্ডওয়্যার সার্কিটের মূল পয়েন্ট (ম্যাম হার্ডওয়্যার জিজ্ঞাসা করলে)

- **18650 Li-ion ব্যাটারি (3.7V):** পোর্টেবল পাওয়ার সোর্স (৩.০V কাট-অফ থেকে ৪.২V ফুল চার্জ)।
- **IP2312 3A Type-C ফাস্ট চার্জার:** সিঙ্ক্রোনাস বাক চার্জার (>৯২% এফিশিয়েন্সি), ৩A রেটিং। সাধারণ TP4056 এর মতো গরম হয় না।
- **SPST সুইচ:** ব্যাটারি পজিটিভ লাইনে ইন-লাইন বসানো; স্ট্যান্ডবাই ড্রেন শূন্য করে।
- **XL6009 4A বুস্ট কনভার্টার:** ব্যাটারির ৩.৭V কে স্টেপ-আপ করে স্থির **5.0V DC** দেয়। W103 (10k) ট্রিমার দিয়ে টিউন করা, যা ESP32 ব্রাউন-আউট বন্ধ করে।
- **রেজিস্টর ডিভাইডার ও Pin 4:** ৫V ও GND এর মাঝে সিরিজে দুটি রেজিস্টর দিয়ে মিডপয়েন্ট **Pin 4 (GPIO 4 / ADC)** এ নেওয়া হয়েছে, যা নিরাপদে ৩.৩V এর মধ্যে ব্যাটারি ভোল্টেজ মাপে।
- **ESP32-CAM পিনআউট:** Pin 1 (5V VCC), Pin 2 (GND), Pin 4 (ADC ট্যাপ)।

DETECTA — VIVA Q&A DEFENSE & TECHNICAL SPECS

ম্যামের সম্ভাব্য ১০টি কঠিন ভাইভা প্রশ্ন ও স্পেসিফিকেশন চিট-শীট

Software Lead: Ahmmed Waseg (251031046)

৪. ম্যামের সম্ভাব্য ৮টি কঠিন প্রশ্ন ও আপনার মোক্ষম উত্তর (VIVA DEFENSE CHEAT SHEET)

প্রশ্ন ১: ESP32-CAM ওয়াই-ফাইতে কানেক্ট হওয়ার সময় বা স্ট্রিমিং করার সময় হঠাৎ রিস্টার্ট দিত? কীভাবে সলভ করেছ?

> **উত্তর:** "ম্যাম, Wi-Fi RF ট্রান্সমিট করার সময় প্রায় ৪৫০mA কারেন্ট স্পাইক হয়, যার ফলে লো-ভোল্টেজ ড্রপ হয়ে ইন্টারনাল RTC ব্রাউনআউট রিস্টার্ট দিত। আমরা দুটি স্টেপে এটি সলভ করেছি: (১) হার্ডওয়্যারে আমরা হাই-কারেন্ট **XL6009 4A বুস্ট কনভার্টার** দিয়ে ভোল্টেজকে ৫.০V এ লক করেছি; (২) সফটওয়্যারে `main.cpp`-এর শুরুতে রেজিস্টার লেভেলে ব্রাউনআউট ডিটেক্টর ডিসেবল করেছি: `WRITE_PERI_REG(RTC_CNTL_BROWN_OUT_REG, 0);`। এরপর থেকে সিস্টেম সম্পূর্ণ স্টেবল।"

প্রশ্ন ২: ভিডিও স্ট্রিমে এত হাই ফ্রেমরেট (20–25+ FPS) কীভাবে পেলে? সাধারণত তো ৮-১০ FPS এর বেশি ওঠে না!

> **উত্তর:** "ম্যাম, ৩টি অপ্টিমাইজেশন করেছি: (১) ESP32-CAM এ 4MB PSRAM দিয়ে **ডুয়াল ফ্রেমবাফারিং** (`CAMERA_FB_IN_PSRAM, fb_count = 2`) করেছি, ফলে এক ফ্রেম ট্রান্সমিট হওয়ার সময় পরবর্তী ফ্রেম মেমোরিতে রেডি থাকে; (২) OpenCV-এর ব্লকিং সকেট বাইপাস করে নিজস্ব মাল্টি-থ্রেডেড পাইথন চাঙ্ক পার্সার লিখেছি; (৩) এআই ইঞ্জিনে **২৫-ফ্রেম স্ট্রাইড ভেরিফিকেশন ক্যাশ** ব্যবহার করায় প্রতি ফ্রেমে ভারী ফেস মডেল রান হয় না, ফলে গড় লেটেন্সি মাত্র ১-২ms এ নেমে এসেছে।"

প্রশ্ন ৩: Dlib ব্যবহার না করে InsightFace ArcFace কেন ব্যবহার করেছ?

> **উত্তর:** "ম্যাম, Dlib এর মডেলটি পুরনো (১২৮-D ভেক্টর) এবং ইউক্লিডিয়ান ডিসটেন্স নির্ভর। মুখ সামান্য বাঁকা থাকলে বা কম আলোতে Dlib ফেস মিস করে। পক্ষান্তরে, **ArcFace** একটি ডিপ কনভোল্যুশনাল মডেল যা **৫১২-ডাইমেনশনাল হাইপার-স্ফেয়ারিক্যাল ফিচার ভেক্টর** তৈরি করে এবং ৫-পয়েন্ট ল্যান্ডমার্ক অ্যালাইনমেন্টের পর কোসাইন সিমিলারিটি দিয়ে ম্যাচ করায় এর নির্ভুলতা ৯৯.৮% এর বেশি।"

প্রশ্ন ৪: খালি আঙুল দেখালে কলম যেন না ভাবে, এই অ্যালগরিদমটা কীভাবে কাজ করে?

> **উত্তর:** "ম্যাম, আমরা YOLO এর পেন বক্সের সাথে MediaPipe এর হ্যান্ড ল্যান্ডমার্কের স্প্যাশিয়াল ইন্টারসেকশন মিলিয়েছি। যদি পেন বক্সটি সম্পূর্ণ খালি আঙুলের চামড়ার সীমানার ভেতরে সীমাবদ্ধ থাকে, তবে সিস্টেম নিশ্চিত হয় যে এটি খালি আঙুল এবং ডিটেকশনটি বাতিল করে। আর যদি আঙুলের মাথা থেকে বাস্তব কোনো অবজেক্ট অন্তত **২৫ পিক্সেল বাইরে প্রসারিত (protruded)** থাকে, তখনই কেবল এটি কনফার্ম করে যে ইউজার হাতে একটি আসল পেন ধরে আছে।"

প্রশ্ন ৫: FastAPI সার্ভারে একাধিক ক্লায়েন্ট ভিডিও দেখলে ল্যাগ করে না কেন?

> **উত্তর:** "ম্যাম, কারণ আমরা **Single-Pass JPEG Caching** করেছি। ভিডিও ফ্রেম প্রসেস ও বাউন্ডিং বক্স আঁকার কাজটি ব্যাকগ্রাউন্ডে একটি সিঙ্গেল এআই থ্রেডে একবারই হয় এবং মেমোরিতে `latest_jpeg_bytes` হিসেবে সেভ থাকে। ১০ জন ক্লায়েন্ট কানেক্ট হলেও সেই সেভ করা মেমোরি থেকে স্ট্রিম যায়, কাউকে নতুন করে এনকোড করতে হয় না। এতে CPU ব্যবহার ৭০% কমে যায়।"

প্রশ্ন ৬: প্রজেক্টে ক্লাউড বা টেলিগ্রাম বট ব্যবহার না করে লোকালি কেন করলে?

> **উত্তর:** "ম্যাম, ক্লাউড-বেজড ক্যামেরায় ভিডিও ফুটেজ থার্ড-পার্টি বিদেশি সার্ভারে যায় যা হ্যাক হতে পারে এবং প্রাইভেসি নষ্ট হয়। এছাড়া ইন্টারনেটে ফ্রেম পাঠাতে গেলে ২০০–৫০০ms লেটেন্সি তৈরি হয়। আমাদের Detecta সম্পূর্ণ **Local-First Architecture**—ইন্টারনেট ছাড়াও লোকাল ওয়াই-ফাইতে সাব-৫০ms লেটেন্সিতে শতভাগ নিরাপদে কাজ করে।"

প্রশ্ন ৭: সার্কিটে Pin 4 (GPIO 4)-এ রেজিস্টর ডিভাইডার কেন দেওয়া হয়েছে?

> **উত্তর:** "ম্যাম, ESP32-CAM এর অ্যানালগ পিন (ADC) সর্বোচ্চ ৩.৩ ভোল্ট পর্যন্ত সহ্য করতে পারে। কিন্তু আমাদের ব্যাটারি বা বুস্ট লাইনের ভোল্টেজ থাকে ৩.৭V থেকে ৫.০V পর্যন্ত। সরাসরি দিলে চিপের এডিসি পুড়ে যেত। তাই দুটি রেজিস্টর দিয়ে একটি ভোল্টেজ ডিভাইডার তৈরি করে ভোল্টেজকে ড্রপ করে ৩.৩V এর নিচে নামিয়ে পিন ৪ এ দেওয়া হয়েছে যাতে নিরাপদে ভোল্টেজ মনিটর করা যায়।"

প্রশ্ন ৮: YOLO11s এ কোন কোন ক্লাস ডিটেক্ট হয়? পার্সেল বা কার্টন বক্স কীভাবে চেনে?

> **উত্তর:** "ম্যাম, আমরা ২১টি কিউরেটেড ক্লাস রেখেছি—মানুষ, ফোন, ল্যাপটপ, বোতল ইত্যাদি। আর বক্স ও পার্সেলের জন্য আমরা COCO ডেটাসেটের ক্লাস ২৪ ও ২৬ (Handbag/Box) এবং ক্লাস ২৮ (Suitcase/Carton/Box) কে ইন্টারনালি রি-ম্যাপ করে 'Box' হিসেবে কনফিগার করেছি। ফলে ডেলিভারি কার্টন এবং স্টোরেজ বক্স অনায়াসে ডিটেক্ট হয়।"

৫. এক নজরে সব টেকনিক্যাল ডেটা ও সংখ্যা (QUICK NUMBERS CHEAT SHEET)

প্যারামিটার (Parameter)	**টেকনিক্যাল ভ্যালু / স্পেসিফিকেশন** (Technical Specification)
ক্যামেরা সেন্সর ও রেজোলিউশন	OmniVision OV2640 (2.0 MP) | VGA (640 × 480), Quality = 16, 20MHz XCLK
স্ট্রিমিং পোর্ট ও মেমোরি বাফার	Dedicated HTTP Port 81 (`/stream`) | 4MB External PSRAM Dual-Buffer (`fb_count = 2`)
লাইভ ফ্রেমরেট ও লেটেন্সি	**20.0 – 24.5 FPS** (Continuous over local Wi-Fi) | **~52 ms** End-to-End Latency
অবজেক্ট ডিটেকশন মডেল	Ultralytics YOLO11s (CUDA FP16 Tensor Cores) | **11.4 ms** Inference Latency
টার্গেট অবজেক্ট ক্লাস সংখ্যা	২১টি কিউরেটেড ক্লাস (মানুষ, বক্স/কার্টন, ল্যাপটপ, ফোন, কলম, বই, কিবোর্ড ইত্যাদি)
ফেস রিকগনিশন মডেল	InsightFace ArcFace (512-D Hyperspherical Vector, Cosine Similarity Threshold: 0.45)
ফেস স্ট্রাইড ক্যাশিং ইনোভেশন	**২৫ ফ্রেম** স্ট্রাইড (Amortized Face Latency drops from 38ms to **<2ms per frame**)
হ্যান্ড জেসচার ও পেন ফিল্টার	MediaPipe Hands (21 3D Landmarks, 7 Gestures) | Spatial Containment + 25px Protrusion
সার্ভার ও ডেস্কটপ আর্কিটেকচার	FastAPI (Single-Pass Cached Shared JPEG) | Microsoft Edge WebView2 (`Detecta.exe`)
পাওয়ার ও ব্যাটারি ব্যাকআপ	18650 Li-ion 3.7V (~12.21 Wh) | IP2312 3A Type-C Charger | XL6009 Boost 5.0V | **~৭.৫ ঘণ্টা ব্যাকআপ**

💡 **শেষ কথা:** প্রেজেন্টেশনের সময় শান্ত থাকবেন এবং সফটওয়্যার লিড হিসেবে আপনার তৈরি করা **YOLO11s, ArcFace 25-Frame Stride Caching, এবং Smart Pen Algorithm**-এর কথা আত্মবিশ্বাসের সাথে বলবেন। শুভকামনা!